

9/8 논문 - Deep Residual Learning for Image Recognition

1. Introduction



논문을 다루고 있는 주제가 무엇인지와 해당 주제의 필요성이 무엇인가
논문을 제안하는 방법이 기존 방법의 문제점에 대응되도록 제안 되었는가

논문이 다루는 분야

CNN을 이용한 이미지 분류.

층을 깊게 쌓을수록 낮은 수준의 경계나 질감부터 높은 수준의 객체 의미까지 계층적으로 표현할 수 있다. 그래서 depth는 성능의 중요한 요소로 여겨졌고, 자연스럽게 이런 질문이 나온다. 그냥 더 쌓으면 되는 것 아닌가?

안 됐다. 이 논문은 그 이유를 파고든다.

해당 task에서 기존 연구 한계점

층을 깊게 쌓으면 학습이 안 되는 문제는 원래 알려져 있었다. gradient가 사라지거나 폭발하는 문제인데, 이건 초기화를 잘 하고 Batch Normalization을 쓰면서 상당 부분 해결됐다.

그런데도 문제가 남았다. 18층 plain network보다 34층 plain network가 더 나뻐다.

여기서 논문이 짚는 지점이 좋다. **test error가 아니라 train error가 높았다.** 과적합이라면 train은 잘 맞추고 test만 틀려야 하는데, 34층은 train조차 못 맞췄다. 외워버려서 생긴 문제가 아니다. 아예 학습이 안 되고 있는 것이다. 논문은 이것 degradation problem이라고 부른다.

왜 이상하냐면, 34층은 18층 뒤에 16층을 더 붙인 것이니까 그 16층이 아무 일도 안 하면 최소한 18층과 같은 성능은 나와야 한다. 그런데 더 나쁘다는 건 네트워크가 아무것도 안 하기조차 못 배우고 있다는 뜻이다.

논문의 contributions

1. $H(x)$ 를 직접 학습하는 대신 $F(x) = H(x) - x$, 즉 입력과의 차이만 학습하도록 문제를 다시 세웠다.

2. shortcut connection으로 $H(x) = F(x) + x$ 를 구현했다. 파라미터가 추가되지 않고 연산량도 거의 늘지 않는다.
3. ImageNet과 CIFAR-10에서 plain network의 degradation을 재현하고, ResNet은 깊어질수록 오히려 좋아짐을 보였다.
4. 152층 모델로 ImageNet top-5 error 4.49%(단일 모델), 앙상블 3.57%를 기록했다. object detection으로도 확장해 다른 task에서도 통함을 보였다.

2. Related Work



Introduction에서 언급한 기존 연구들에 대해 어떻게 서술하는가
제안 방법의 차별성을 어떻게 표현하고 있는가

residual을 쓰는 발상 자체가 새로운 건 아니다. VLAD와 Fisher Vector는 기준 벡터와의 차이를 인코딩했고, 수치해석의 multigrid 방법도 전체 해를 한 번에 구하는 대신 스케일 사이의 residual을 푼다. 전체를 직접 구하는 것보다 기준점에서의 변화량을 구하는 편이 쉽다는 직관은 이미 여러 분야에 있었다.

shortcut connection도 마찬가지로 MLP, auxiliary classifier, Inception 등에서 쓰여 왔다.

차별점은 Highway Network와 비교할 때 분명해진다. Highway는 학습 가능한 gate로 정보를 얼마나 통과시킬지 조절한다. gate가 닫히면 shortcut이 없는 것과 같아진다. 반면 ResNet의 identity shortcut은 파라미터가 없고 항상 열려 있다. 입력 x 는 무조건 그대로 전달되고, residual branch는 거기에 더할 변화량만 학습한다.

3. 제안 방법론



Introduction에서 언급된 내용과 동일하게 작성되어 있는가
Introduction에서 언급한 제안 방법이 가지는 장점에 대한 근거가 있는가
제안 방법에 대한 설명이 구현 가능하도록 작성되어 있는가

Main Idea

블록의 출력은 이렇게 정의된다.

$$H(x) = F(x, \{W_i\}) + x$$

x 는 블록의 입력이고, F 는 conv 두세 층이 학습하는 residual이다.

왜 이게 쉬워지는지가 이 논문의 전부다. 지름길이 없으면 블록이 입력을 그대로 통과시키기 위해 $F(x) = x$ 를 만족해야 한다. 그러려면 여러 conv 층의 가중치가 아주 특정한 값이어야 하고, 랜덤한 초기값에서 그걸 찾아가는 건 어렵다. 지름길이 있으면 같은 일을 하는 데 $F(x) = 0$ 이면 된다. 가중치를 전부 0으로 보내면 되는데, 초기값이 애초에 0 근처인 데다 weight decay가 계속 0쪽으로 밀어주니까 훨씬 쉽다.

같은 목표인데 도달해야 할 가중치가 완전히 달라진 것이다.

차원이 안 맞을 때는 이렇게 처리한다.

- 공간 크기가 줄어드는 경우: shortcut에도 stride를 걸어 맞춘다.
- 채널 수가 다른 경우: Option A는 모자란 채널을 0으로 채운다(파라미터 없음).
Option B는 1x1 convolution으로 채널을 변환한다. $y = F(x) + W_s \cdot x$

50층 이상은 bottleneck 블록을 쓴다. 1x1로 채널을 줄이고, 3x3을 좁아진 채널에서 수행하고, 다시 1x1로 복원하는 구조다. 3x3 convolution은 커널 크기 때문에 1x1보다 9배 비싸고 채널 수는 제곱으로 들어오기 때문에, 비싼 연산을 좁은 구간에서만 하면 비용이 크게 준다. 256채널 bottleneck이 64채널 기본 블록과 비슷한 연산량이 되고, 덕분에 같은 예산으로 채널을 4배 넓게 쓸 수 있다.

Contribution

Option A/B/C 비교가 이 논문에서 제일 잘 설계된 부분이라고 생각한다.

Table 3(10-crop 기준)을 보면 plain-34가 top-1 error 28.54%인데, 파라미터를 하나도 안 쓰는 Option A만으로 25.03%까지 내려간다. 모든 shortcut에 projection을 쓰는 Option C는 24.19%로 조금 더 좋지만 차이가 1%p 이하다.

이 배치가 왜 중요하냐면, projection을 쓸 때만 잘 됐다면 residual 구조가 좋은 게 아니라 파라미터를 더 넣어서 좋아진 거라는 반박이 가능하다. A가 잘 되니까 그 반박이 막힌다. degradation을 푸는 건 파라미터가 아니라 지름길이 있다는 사실 자체다.

실제 모델은 B를 쓴다. projection이 들어가는 곳이 차원 바뀌는 지점 몇 군데뿐이라 파라미터 증가가 1%도 안 되는데, 그 정도면 안 쓸 이유가 없다. bottleneck에서는 채널이 4배로 뛰기 때문에 A로 하면 shortcut의 75%가 0이 되어버리는 것도 이유다.

layer response 분석도 가설과 맞는다. ResNet의 residual branch 출력이 plain보다 전반적으로 작았고, 깊은 모델일수록 각 층이 신호를 더 조금씩만 건드렸다. identity를 기준으로 두고 필요한 만큼만 수정한다는 설명과 일치한다.

구현은 convolution 직후 activation 전에 BN을 넣었고, SGD에 batch size 256을 썼다. dropout은 쓰지 않았다.

4. 실험 및 결과



Introduction에서 언급한 제안 방법의 장점을 검증하기 위한 실험이 있는가

Dataset

- ImageNet 2012: 1,000 클래스, train 128만 장. 메인 실험.
- CIFAR-10: 10 클래스, train 5만 장. 20층부터 1,202층까지 늘려가며 depth에 따른 변화를 관찰.
- PASCAL VOC 2007/2012, MS COCO: detection으로 옮겨도 통하는지 확인.

Baseline

- plain network: VGG 방식으로 3x3 convolution만 쌓고 shortcut은 없는 18층/34층. ResNet과 depth, width, 파라미터 수, 연산량을 맞춰서 shortcut 유무만 차이나게 했다.
- shortcut ablation: Option A/B/C 비교.
- 기존 모델: VGG-16/19, GoogLeNet, BN-Inception 등 당시 SOTA.
- detection: Faster R-CNN에서 backbone만 VGG-16과 ResNet-101로 교체.

결과

1. degradation 재현

plain-34가 plain-18보다 train, validation error 모두 높았다. BN을 써서 gradient가 정상적으로 흐르는 상태였으므로 vanishing gradient로는 설명되지 않는다.

2. 뒤집힘

ResNet에서는 34층이 18층보다 좋았다. 같은 34층끼리 비교하면 plain 대비 top-1 error가 약 3.5%p 낮다. 층 수와 파라미터가 같고 shortcut만 다르니까 원인이 구조에 있다는 게 분명해진다.

3. 깊이 확장

50, 101, 152층으로 갈수록 계속 좋아졌다. 152층은 11.3B FLOPs로 VGG-19의 19.6B보다 오히려 적다. 8배 깊은데 연산량은 60% 수준인 건 bottleneck 덕분이다. top-5 validation error 4.49%, 앙상블은 test error 3.57%.

4. 1,202층

CIFAR-10에서 110층이 test error 6.43%인데 1,202층은 7.93%로 나빠졌다. 그런데 1,202층의 train error는 0.1% 미만이다. 학습은 완벽하게 됐는데 test만 나쁘다.

이건 앞의 degradation과 정반대 성격의 실패다. degradation은 train부터 안 되는 최적화 문제였고, 여기는 train은 되는데 test가 안 되는 과적합이다. CIFAR-10은 5만 장인데 1,202층 모델은 파라미터가 1,900만 개 정도라 데이터에 비해 모델이 지나치게 크다. 논문도 regularization을 따로 쓰지 않았다고 언급한다.

5. detection

COCO에서 backbone만 ResNet-101로 바꿨더니 mAP가 21.2에서 27.2로 올랐다. 백본 교체만으로 6포인트는 큰 폭이고, 이후 detection 모델들이 ResNet을 기본 백본으로 쓰게 된 이유가 여기 있다.

5. 결론 (배운점)



연구의 의의 및 한계점, 본인이 생각하는 좋았던/아쉬웠던 점 (배운점)

의의

이 논문이 한 일은 학습시킬 함수를 다시 정의한 것이다. $H(x)$ 를 처음부터 만들지 말고, 입력을 기준으로 두고 얼마나 바꿀지만 배우게 했다. 덧셈 하나 추가한 게 전부인데 34층에서 152층까지 갈 수 있게 됐다.

plain network와 조건을 맞춰서 비교한 것도 좋았다. 깊어져서 좋아진 건지 residual 때문에 좋아진 건지가 섞이지 않는다.

한계

- residual로 바꾸면 왜 최적화가 쉬워지는지에 대한 이론적 설명은 없다. layer response가 작더라는 관찰이 근거의 대부분인데, 가설과 어긋나지 않는다는 정도지 증명은 아니다.
- 1,202층 결과를 그대로 실은 건 오히려 좋았다고 생각한다. 최적화가 풀렸다고 좋은 모델이 되는 건 아니라는 게 같은 논문 안에서 드러난다.
- addition 다음에 ReLU가 오는 구조라 identity path가 끝까지 그대로 흐르는 않는다. 후속 논문에서 정리된 걸로 알고 있다.

배운 점

읽으면서 제일 오래 막혔던 건 $F(x) = 0$ 을 학습한다는 말이었다. 아무것도 안 하는 걸 배운다는 게 무슨 뜻인지 감이 안 왔다.

정리하고 나니 순서가 반대였다. 원래도 "아무것도 안 하기"는 필요한 동작이었는데, 지름길이 없을 때는 그게 어려운 정답이었던 것이다. 여러 층을 통과시켜서 입력을 그대로 복원하

려면 가중치가 특정한 값이어야 하니까. 지름길을 놓으면 같은 동작의 정답이 0으로 바뀐다. 문제를 쉽게 만든 게 아니라 정답을 쉬운 위치로 옮긴 것에 가깝다.

Option A를 왜 실제로는 안 쓰는지도 한참 걸렸다. 파라미터가 안 드는데 성능도 거의 같으면 A가 나은 것 아닌가 싶었다. A는 실전용 설계가 아니었다. 파라미터 없이도 degradation이 풀린다는 걸 보여주려고 만든 조건이지 쓰라고 만든 게 아니었다.

표를 볼 때 train과 test를 갈라서 봐야 한다는 것도 배웠다. 이 논문에는 실패가 두 번 나오는데 성격이 완전히 다르다. 34층 plain은 train부터 안 됐고, 1,202층은 train만 됐다. 앞은 구조를 고쳐야 하고 뒤는 데이터나 regularization을 봐야 한다. 결과가 나쁘다는 것만 보고 원인을 하나로 뭉뚱그리면 안 되겠다.